Bulgarian Diploma Thesis

Big Break! - Java Film Script Breakdown Tool

Vili-An Atanasova Doneva, 200200140

Student: Vili-An Atanasova Doneva, Date _____

signature

Supervisor: _____, Date _____

signature

Department of Computer Science, AUBG

Blagoevgrad, 2025

Declaration template for Senior Project and Diploma Thesis

Title: Big Break! - Java Film Script Breakdown Tool

Author: Vili-An Atanasova Doneva

Abstract:

Big Break! is a Java tool for automating film script breakdown using Natural Language Processing (NLP). The software requires the user to input a script draft as a .txt file. Then, it analyses the text and generates a script schedule – a file including the shoot days, scenes per day, characters, day length, time of day, etc. To extract the characters for each scene, the software uses a complex heuristic logic combined with a pre-trained NER (Named Entity Recognition) model. *Big Break!* additionally uses a trie data structure to compare names to the common names file. The program performs clustering. It uses the Levenshtein distance and Jaro-Winkler algorithm and a Disjoint Set Union data structure to merge the cleaned data entries that belong to the same entity (e.g. *Nelly* and *Nelly Smith*). *Big Break!* relies on alias canonicalization, which standardizes the names (e. g. *Nelly V.O.* to *Nelly*). It solves the Resource-Constraint Project Scheduling Problem (RCPSP) with a Serial Schedule Generation Scheme (SSGS).

Declaration of authorship:

“The Senior Project/Bulgarian Diploma Thesis presented here is the work of the author solely, without any external help, under the supervision of Zachary Hutchinson. All sources, used in development, are cited in the text and in the Reference section.”

Author: Vili-An Atanasova Doneva

1. Introduction

The main task of an Assistant Director (AD) during a film's pre-production process is to extract information from each scene and turn it into a shooting schedule. A script breakdown includes all the scene numbers, headings, locations, times of day, characters, etc. and is used to make a production schedule. Script revisions can vary from 3-4 in small productions to 15+ in big productions before the start of the shooting and even more after. This wastes a lot of time reading revised pages and trying not to miss any important details. Having an already composed, editable schedule with all the broken-down information for each script revision facilitates the process. This allows the 1st AD to focus on making the Director's vision happen rather than being stuck in front of the screen.

The *Big Break!* software allows the user to input a .txt film script file. It divides the script into scenes and collects all their numbers, headings, locations, and times using regular expression (Regex) patterns and rules. Everything between two headings is treated as content, which is used for the actual Natural Language Processing (NLP). All the dependencies are managed with Maven.

The characters are extracted through heuristic methods, regex, and the pre-trained Named Entity Recognition (NER) model – NameFinderME. What is used depends on the types of names encountered – speaker cue name, in-line name (part of the dialogue or the scene description), names which are not proper nouns (*OLD MAN, RECORDED VOICE,* etc.), and names when a character is first introduced (*This is Nelly (23)…)*. Additionally, the proper nouns found are compared using a Trie data structure to a .txt file with various common names. This boosts the confidence of the match.

Most cases are covered by the heuristic logic, which normalizes the names and analyzes them based on regex patterns. However, in-line names are more difficult to extract as they have no distinct position or format. This is why NameFinderME or the first-introduced logic is used. NameFinderME is a class in the Apache OpenNLP library, which is an open-source NLP Java toolkit. It

uses the Maximum Entropy algorithm internally to predict named entities like the character names.

For all character names, the program performs clustering and alias canonicalization. Clustering groups together all the character names that belong to the same entity using Disjoint Set Union (Union-Find) data structure, custom logic and the Levenshtein distance algorithm and Jaro-Winkler algorithm to group names with small differences together. Canonicalization standardizes the data by removing parentheticals and normalizes titles and maps the different entries together under one canonical name through heuristics.

For props and set dressings, the program uses Parts-Of-Speech (POS) Tagger to determine the grammatical role of each word in the script. The POS Tagger also detects patterns of grammatical roles in a sentence (e.g. verb -> determiner -> noun). This allows heuristic rules to facilitate the detection of props and set dressings.

Big Break!'s scheduling depends on various factors – location, time, maximum pages per day, scene order, character overlaps, load per day, etc. This causes the Resource-Constraint Project Scheduling Problem (RCPSP) with a Serial Schedule Generation Scheme (SSGS).

The program uses Spring Boot to connect through a REST API to the React frontend. Having the visual aspect of *Big Break!* would allow the user to easily interact with the software without burdening them with heavily technical processes.

The reason why I chose to pursue *Big Break!* as my senior project is majorly connected to my interest in film and my belief that technology can save time. The most prominent lesson I learned during my time in the film studio is that time and money are closely connected in a film production. Works like mine can help large films save time, money, and focus on the creative aspects of moviemaking.

2. Specification of the Software Requirements and their Analysis

Functional Requirements

- The user will be able to choose only a .txt file.
- The user will be able to view the shooting schedule.
- The user will be able to view each scene number, heading, location, and time of day (if available).
- The user will be able to view the characters of each scene.
- All names pointing to the same character will appear only as the common canonical name.
- The schedule will be editable.
- The system will be able to detect props and set dressings based on heuristic rules working with grammatical roles.

Non-Functional Requirements

- The system will be able to complete script parsing and entity extraction within 20 seconds on a machine with 16 GB RAM.
- At least 80% of the characters will be accurately extracted.

Analysis of (Some for Now) Functional Requirements

The user will be able to view each scene number, heading, location, and time of day (if available).

The extraction of the scene numbers, headings, locations, and times of day happens in the ScriptParser.java file, which separates all script scenes. It uses helper methods - *loadScript (String filePath)* to access the script .txt file and to normalize spaces and breaks, and removePageNumbers (String script) to remove page headers/footers and obvious page numbers. The primary method is List <Scene> splitScenes(String script). It calls all the other two methods, splits the scenes and returns a list of all scenes, filling up the Scene class constructor with scene order number (the actual number of the scene in the order they appear), scene script numbers (the scene number written in the script), headings, locations, and times (if available). In order to find the values, it uses Regex patterns and rules depending on the different types of headings – scene number on a separate line or a scene number in line with the heading. Here is a pseudocode explanation of the Regex patterns:

```
// RegEx helpers
// numberOnly: line that contains only a page/scene number, e.g. "98" or "98A"
// headingPattern: matches an optional inline number + location keyword (INT/EXT/...) +
   optional description + optional time
// headingKeyword: matches a heading keyword at the start of a line (INT./EXT./etc.)
```

Figure 1: Explanation of the Regex patterns in splitScenes (String script)

Pseudocode of adding a scene with the Scene constructor, incrementing the scene order number, and returning all the scenes:

```
if currentHeading is not null:
    sceneRawCounter = sceneRawCounter + 1
    add Scene(sceneRawCounter, currentSceneNumber, currentHeading, trim(currentContent),
    currentLocation, currentTime) to scenes

return scenes
```

Figure 2: Pseudocode of adding a scene and returning all the scenes

Later, all the obtained values (possibly, besides the scene order number) will be displayed in the UI.

The system will be able to detect props and set dressings based on heuristic rules working with grammatical roles.

The POS Tagger of Apache OpenNLP automatically determines grammatical roles. It once again uses the Maximum Entropy algorithm internally, e. g.:

String[] sentence = {“Elphaba”, “looked”, “in”, “the”, “mirror”, “.”};

String[] tags = posTagger.tag (sentence);

Result:

Elphaba/NNP (proper noun) looked/VBD (verb) in/IN (preposition) the/DT (determiner) mirror/NN (noun) ./.

After having the grammatical data, the software uses it as input to the heuristic rules that determine whether a noun is a prop or a set dressing. For example, if the noun is preceded by a manipulation verb (grab, pick up, take, etc.), then it is most probably a prop. However, if the previous word is a preposition (on, in, behind, etc.), then the noun is most probably part of the set dressing category. Adding a NameFinderME NER model here will improve the performance further.

The user will be able to view the characters of each scene.

This will be updated further:

The characters are found in the script using several extractor methods: *extractSpeakerCues, extractInlineName, extractPersonWord, and extractIntroCharacters*. Several helper methods are also created – *normalizeName, normalizeNameInline, isCharacterCue, isStopPhrase, and safeSnippet*.

The *extractSpeakerCues* method is responsible for finding the speaker above dialogue names. However, it covers two cases – uppercased names with parentheses and without them.

Pseudocode for part of the isCharacterCue(String line) method that is mainly responsible for obtaining characters on the initial heuristics side:

```
40      // 6. Heuristic: most tokens must be name-like (>= 80%) and there can't be too many tokens (<=5)
41      if nameTokens >= ceil(tokenCount * 0.8) and tokenCount <= 5 then
42          // reject lines containing punctuation that indicates stage directions or transitions
43          if trimmed contains any of [! ? , . ; : ( ) [ ] ] then
44              return false
45          end if
46
47          // 7. Reject if any meaningful uppercase token is in the blacklist or stage-verb lists
48          for each tok in TextUnits.tokenize(trimmed) do
49              up = removeNonLettersAtoZ(tok)   // keep only A-Z chars
50              if up is empty then continue
51              if BLACK_LIST contains up then return false
52              if STAGE_VERBS contains up then return false
53          end for
54
55          // Passed all tests – it's a character cue
56          return true
57      end if
58
```

Figure 3: Pseudocode for isCharacterCue (string line)

The pseudocode shows the main checks of whether the name tokens are at least 80% of all tokens in the line and whether the token count is less than or equal to five, so it could be considered a character cue line. In step 7 in *Figure 3*, the software ensures that each token in the line is not part of the BLACK_LIST or STAGE_VERB words. BLACK_LIST contains frequent movie sounds, movie slang, and non-name words such as *are, in, a/an,* etc. The STAGE_VERB set relates to words for camera movement that can be included in the original script.

The extractInlineNames covers the scenarios of names in a dialogue or scene descriptions. It is using the pre-trained model NameFinderME.

The extractPersonWord extracts characters which are not proper names for characters that are unnamed. An example of that is *OLD WOMAN*. However, it checks different regexes based on whether the human-related word is an adjective, noun, or voice connected word, positioned above dialogue.

The extractIntroCharacters method handles situations where the characters are first introduced, as they usually have specific formatting. This is done through matching regex patterns.

The characters are extracted, clustered, and a canonical name is chosen for all. Then, only the unique characters for each scene are reset in the Scene class. This makes them easily viewed in the schedule. However, they would have a specific confidence score threshold they need to pass through because some of the NameFinderME findings are occasionally inaccurate. Nonetheless, the goal is also for them to be editable and for the director to see the ones that did not pass the threshold as suggestions. Also, new characters could be added manually by the Assistant Director.

3. Design of the Software Solution

Algorithms

Levenshtein (Edit) Distance

The Levenshtein (edit) distance algorithm measures the difference between two character sequences. In the program, the algorithm helps perform clustering of similar names, props, or set dressings together that likely refer to the same character or object. The algorithm processes the minimum number of changes needed to turn one string into another. The edits can be:

- Insertion (adding a character): “CAT” -> “COATS”

    - Distance = 2
- Deletion (deleting a character): “SARAH” -> “SARA”
    - Distance = 1
- Substitution (replacing a character): “MONEY” -> “HATER”
    - Distance = 4

Thresholds can be applied so the algorithm can cluster words with custom distances as part of the same group or exclude them. Here is the pseudocode of the essential step of the Levenshtein distance algorithm – filling the matrix:

```
12      // Step 3: Fill matrix
13      for i from 1 to len(a):
14          for j from 1 to len(b):
15
16              if a[i-1] == b[j-1]:
17                  cost = 0    // same letter → no change
18              else:
19                  cost = 1    // different letter → substitution
20
21              dp[i][j] = min(
22                  dp[i-1][j] + 1,        // deletion
23                  dp[i][j-1] + 1,        // insertion
24                  dp[i-1][j-1] + cost    // substitution
25              )
26
```

Figure 4: Pseudocode of the Levenshtein distance algorithm

To explain this piece of code, a is the sequence of characters for the string that the system is going to turn into the sequence of characters for the b string. So, the code starts by checking if the two currently compared characters are the same (lines 16-19). If the letters are the same, the cost of the change variable is 0. Similarly, if the letters are different, the change is 0. This variable is used only for substitution.

	“”	C	A	T
“”	0	1	2	3
C	1	0	1	2
A	2	1	1	2
T	3	2	2	**1**

In the next few lines (21-25), the algorithm calculates *dp* corresponding to which operation – deletion, insertion, or substitution - would require the least number of edits. For deletion, the algorithm uses $\mathrm{dp}[\mathrm{i}-1][\mathrm{j}]+1$. This presents the option of deleting the current character of ***a*** – $[i-1]$ which leaves the system

Table 1: Levenshtein distance algorithm example matrix

having to turn the sequence of characters up to $[i-1]$ in ***a*** to the sequence up to $[j]$ in ***b***. The algorithm gets the value of the position above the current and adds cost 1 for the deletion.

The insertion in the algorithm follows similar logic – $dp[i][j-1]+1$. The system needs to insert into the sequence of characters up to $a[i]$, the last character of the other string, which is $b[j]$. The algorithm removes the last character from ***b*** as a preparation to insert it into the ***a*** sequence – $[i][j-1]$. Finally, it adds one for the cost of insertion, which makes $dp[i][j-1]+1$ the insertion formula.

Substitution follows the following formula using the *cost* variable previously mentioned - $dp[i-1][j-1]+cost$. It evaluates the cost to match ***a*** up to $[i-1]$ to ***b*** up to $[j-1]$. After, it adds the cost of turning $a[i]$ into $b[j]$ that was evaluated in lines 16-19 of *Figure 4*.

For example, if comparing *CAT* and *CUT* as in *Table 1*, here are the values of the first letters' comparison ("C" -> "C"):

Deletion: $dp[i-1][j]+1=dp[1-1][1]+1=dp[0][1]+1=1+1=2$

Insertion: $dp[i][j-1]+1=dp[1][1-1]+1=dp[1][0]+1=1+1=2$

Substitution: $dp[i-1][j-1]+cost=dp[1-1][1-1]+0=dp[0][0]+0=0+0=0$

The final “C” -> “C” distance: $dp[i][j] = \min(2,\ 2,\ 0) = 0$

All other character sequences are compared in the same manner. The last cell of the matrix table (can be seen in *Table 1*) represents the minimal number of edits needed to turn the entire sequence *a* to *b*. The algorithm has already explored all the different paths to determine each value of the matrix as the minimal edit cost. This makes the last value a product of all the minimal edits, resulting in having the ultimate value of the matrix as the total distance of changing *a* into *b*.

Jaro-Winkler similarity

Jaro-Winkler is an algorithm for measuring the similarity of two strings – character names. It considers the common letters and whether they fit in the same position in both words or not. Also, it boosts similarity if both strings have the same prefix up to a defined maximum length. The Jaro-Winkler output varies between 0 (completely different strings) and 1 (equal strings). It is based on the Jaro similarity algorithm, including the Winkler prefix adjustment. The time complexity of the algorithm is *O (N * M)*, where *N* is the length of one string, and *M* is the length of the other.

The algorithm starts with lowercasing both of the strings, so they are easily comparable. It checks if the strings match completely and returns 1. However, in most cases, the algorithm continues. In order for the letters to match, they need to be in a similar position in both strings. This is why the algorithm calculates the maximum distance two characters (s1 and s2) could be apart, and still be considered a match:

$$\text{matchDistance} = \left\lfloor \frac{max(|s1|, |s2|)}{2} \right\rfloor - 1$$

To store the letter matches for both strings, two arrays are set. First, the algorithm loops through the characters of the first string. It sets the start and the end of the search in the other string:

```
23          //loops through characters of s1
24          for (int i = 0; i < s1.length(); i++) {
25              //limits the search only to the allowed matching distance
26              int start = Math.max(0, i - matchDistance);
27              int end = Math.min(i + matchDistance + 1, s2.length());
```

Figure 5: Start and End Regarding the Match Distance

On line 26, for the start, it chooses the maximum value from 0 and the match distance subtracted from the current value. The end of the search is the minimum length of the second string and the current position, added to the match distance plus 1 (so it is included in the search).

The second loop is to iterate through the second string. It checks if the character is already matched and if it is the same character. If true, the arrays are updated to record the match.

The characters which are not matched but not in the same position in both words are called transpositions. To find them, a new loop iterates through the first string. It skips if the character is not matched. If the match exists, it searches for another matched string in the other boolean array. If the characters are not the same, then the transposition count increases.

In order to find the Jaro similarity, the following formula is used:

$$\frac{1}{3}\left(\frac{m}{|s1|} + \frac{m}{|s2|} + \frac{m - t}{m}\right)$$

- m is the number of all matching characters.
- t is half of the number of transpositions
- $|s1|$ and $|s2|$ are the lengths of both strings.

Then, the algorithm considers the prefix. A counter is set for the prefix letters that match both characters. Then, we loop through the first 4 characters or the shorter string if it contains fewer than 4 letters. Every time a match is made, the counter is increased.

This leads to the Jaro-Winkler similarity calculation:

$$SW = SJ + P * L * (1 - SJ)$$

- SW is the Jaro-Winkler similarity.
- SJ is the Jaro similarity.
- P is the scaling factor (0.1 by default).
- L is the length of the matching prefix, which should be up to a maximum of 4 characters.

Data Structures

Trie (Prefix Tree)

In *Big Break!*, the Trie (Prefix Tree) data structure is used to compare extracted names to common first and last names. It avoids repeated linear searches over large name lists and supports fast prefix-based lookup.

The root node acts as a starting point and does not store any character (as in a letter). A map is created to connect a character to its next node in the tree. A Boolean *isWord* variable is introduced to track whether the path from the root to the current node forms a complete, valid word. The operations executed are insert and contain.

In the *insert* function, the system loops through each letter of the inserted word. First, the root node has no characters associated with it, and its *isWord* is *false*.

Then, each character is sequentially processed. If the character has not already occurred in the Trie, a new node is attached to and returned. If the character already exists, the system returns it. The Trie structure uses common prefixes and stores fewer characters. After the last character, the node is marked with *isWord* being *true*. The time complexity of this operation is O(n), where n is the length of the word to insert, and the auxiliary space it takes is O(n) as well. Here is an example of inserting *Mary* and *Mari:*

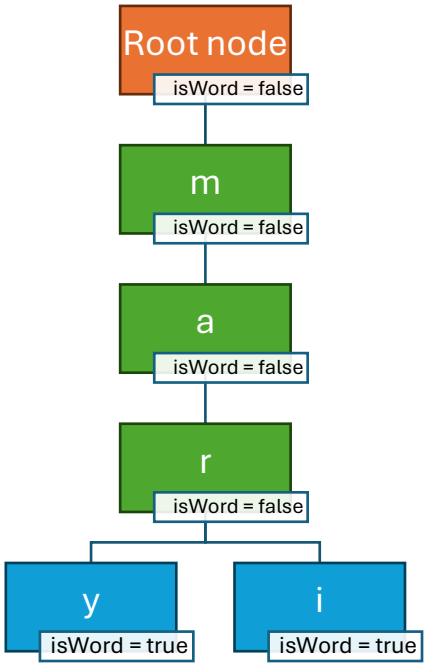

Figure 6: Prefix tree Insertion

In the *contain* function, we check whether the character name is in the Trie structure. The software starts at the root of the prefix tree. It checks if any of its child nodes will match the first character of the name. If true, it moves to the next node and iterates through the following name characters. In case there is no child match for a character, it returns *false* and stops the search. After there are no characters left in the name and all characters have been matched, *Big Break!* checks if the last node reached in the Trie is marked by *isWord* = *true* (the end of a word). Depending on that, it returns either *true* or *false*. The *contain* operation has O(n) time complexity, where n is the length of the word to search, and it takes O(1) auxiliary space.

In *Big Break!,* the Trie structure is used to validate names based on common first and last names. If it matches, it boosts the confidence of the match. However, the first name's match boosts more than the last name's as it is more common, and the database contains more variety.

Union-Find (Disjoint Set Union)

The software uses the Union-Find (Disjoint Set Union - DSU) as a graph clustering structure. It searches for whether two names should belong to the same cluster based on how similar they are. The process starts with each character name being its own parent in a separate set. DSU uses two distinct operations – find and union.

The *find* function searches for the representative (parent) of the set of a given name. This is the root of the parent pointer tree. It is implemented by recursively traversing the parent array until we find the root node (parent of itself):

Figure 7: Find Function

The *union* function combines two sets into one. It takes the indices of two names as input and finds their set representative using the *find* operation. It puts the second pointer tree under the root of the first one:

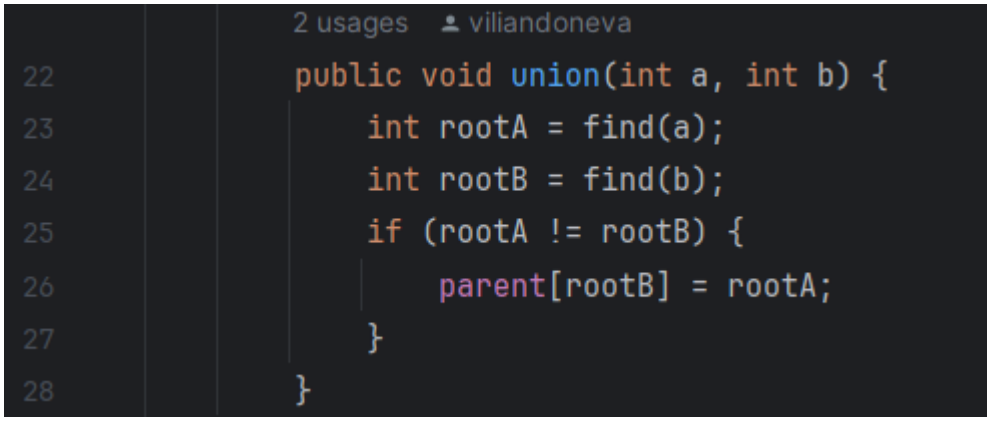

Figure 8: Union Function

Every pair of names is compared. First, the software checks if a name token of one name is contained in the other. If this is not the case, it checks for similarity and difference between the names using the Levenshtein distance and the Jaro-Winkler algorithm. A threshold value is created in order to catch small typos or name variations. This implementation of DSU makes the similarity transitive, meaning that if A is similar to B and B is similar to C, A and C will be part of the same cluster.

After all unions are set, the clusters are extracted from the parent pointer tree. The root of each name node is found. If a cluster with this root exists, the name node is added to the cluster. If no such cluster is found, a new one is created.

Lastly, a canonical name is chosen to have a standardized way to refer to each character. The longest name is chosen because it is often the most descriptive and unlikely to overlap with another in the script.

Inheritance

I have planned to implement inheritance of an ExtractableItem parent class and Character, Prop, and SetDressing as child classes sharing a lot of similar data and methods. Here is a very basic visual of their relationship (*Figure 9*):

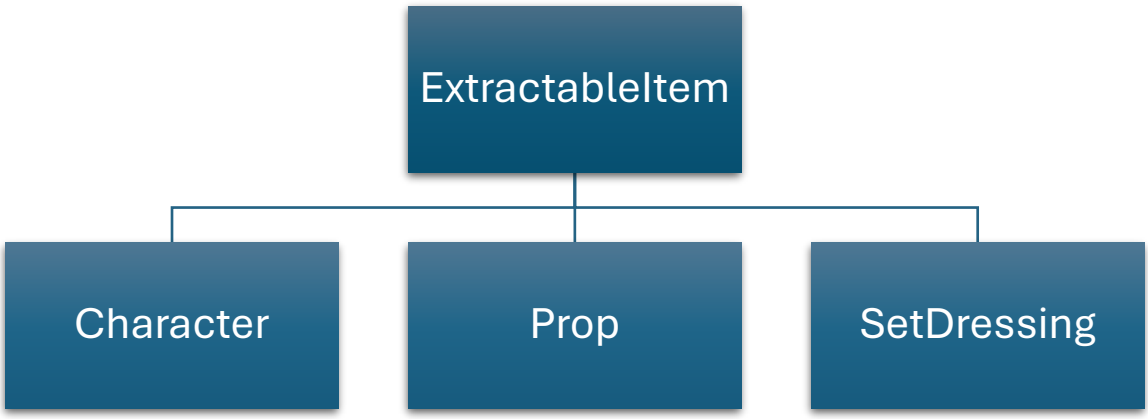

Figure 9: Inheritance

Pipeline

The pipeline in *Big Break!* represents the flow of data and processing stages from the raw script into structured information for all scenes, characters, props, and set dressing. Here are all the stages, summarizing their input, process, and output:

№	Stage	Input	Process	Output
1	Input	.txt script	Clean + normalize	Raw text
2	Scene Split	Raw text	Regex parsing	List of scenes
3	Extraction	Scene text	NER + heuristics	Items (characters, props, set dressings)
4	Clustering	Item features	Similarity grouping	Entity clusters

5	Canonicalization	Raw items	Merge duplicates	Unified items
6	Scheduling	Structured scenes + canonical entities	RCPSP (SSGS heuristic)	Shooting Schedule (grouped by day)
7	Output	Shooting schedule	Export / visualize	UI

Table 2: Pipeline

4. Implementation

Why use Java?

The programming language I chose for the project is Java because it integrates Apache OpenNLP well, as Apache is native to Java. It provides a lot of pre-built models and tools for complicated or tiresome cases.

Java has a strong object-oriented design and has a class-based OOP structure. This helps with clean encapsulation for components like clustering and scene parsing. It also allows the use of interfaces and abstract classes.

Java runs faster than most of the interpreted languages for processing of large amounts of text. It compiles to bytecode and runs on the JVM, allowing for larger film scripts to be processed in a timely manner.

External frameworks/libraries

- **Apache OpenNLP**

  Big Break! uses Apache OpenNLP for tokenization, POS tagging, and handling in-line characters. It allows the exploration of intricate

algorithms (such as the Levenshtein distance algorithm and Jaro-Winkler similarity algorithm) and heuristics on top of it

What is more, it integrates very well with a Java + Maven project. As previously discussed, OpenNLP is Java-native, and this provides stable, production-ready implementations for low-level tasks. Having a standard library decreases the variability of homegrown tokenizers/taggers.

- **React**

  Big Break! uses React for its frontend. One of the reasons is that React's component model (buttons, lists, token-highlighters) can map directly the needed UI pieces.

  React state + controlled components can show model prediction. They can also highlight low-confidence candidates and allow the user to accept, edit, and delete.

  React allows for clean separation between the frontend and the backend. React talks to it through REST endpoints. This helps *Big Break!* create a modern full-stack architecture.

7. References

Carter. "Levenshtein Distance Explained." *YouTube*, YouTube, 26 Jan. 2025, www.youtube.com/watch?v=eneSE4vVAOs.

"Class Levenshteindistance." *LevenshteinDistance (Apache Commons Text 1.14.0 API)*, commons.apache.org/proper/commons-text/apidocs/org/apache/commons/text/similarity/LevenshteinDistance.htm

Jurafsky, Dan, and James H. Martin. *Speech and Language Processing an Introduction to Natural Language Processing, Computational Linguistics,*

and Speech Recognition with Language Models Daniel Jurafsky, James H. Martin. 3rd ed., Stanford University, 2025.

kartik. “Introduction to Disjoint Set (Union-Find Data Structure).” *GeeksforGeeks*, GeeksforGeeks, 17 Jan. 2026, www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/.

Kundu, Arnab. “Jaro and Jaro-Winkler Similarity.” *GeeksforGeeks*, GeeksforGeeks, 16 May 2025, www.geeksforgeeks.org/dsa/jaro-and-jaro-winkler-similarity/.

VR. “Trie Data Structure.” *GeeksforGeeks*, GeeksforGeeks, 18 Jan. 2026, www.geeksforgeeks.org/dsa/trie-insert-and-search/.